Database Design & SQL Interview Guide: Basic to Advanced

> *[!TIP] This guide progresses from foundational concepts to advanced database development tasks, specifically targeting the Lawbase property tax system requirements mentioned in the job description.*

Level 1: Database Foundation (Beginner)

Entities, Attributes, and Keys

- **Entity:** A real-world object (e.g., `Owner`). Becomes a **Table**.
- **Attributes:** Details of an entity (e.g., `FirstName`). Becomes **Columns**.
- **Primary Key (PK):** A unique, non-null identifier for a row (e.g., `OwnerID`).
- **Foreign Key (FK):** A column referring to a PK in another table, creating a relational link.

Relationship Types

1. **One-to-One (1:1):** Very rare. E.g., An `Employee` has one `Desk`.
2. **One-to-Many (1:M):** Most common. E.g., A `Department` has many `Employees`.
3. **Many-to-Many (M:M):** E.g., A `Student` takes many `Courses`. Requires a third "junction table" to link them together in a relational database.

Database Relationships Diagram

Level 2: Structuring Data (Intermediate)

Database Normalization

Normalization reduces redundancy (duplicate data) and protects data integrity (making sure data doesn't conflict when updated).

- **1st Normal Form (1NF):** Eliminate repeating groups. Every cell gets one distinct value.
- **2nd Normal Form (2NF):** Eliminate partial dependencies. All non-key attributes must depend on the *entire* primary key (mostly applies to tables with composite primary keys).
- **3rd Normal Form (3NF):** "Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key." Move data that applies to external concepts into its own table (e.g., don't put `City` and `State` string names directly in the `Owner` table if you already track `ZipCode`. City and State depend on the ZipCode! They belong in a ZipCode table).

Database Normalization Rules vs Spaghetti Data

Level 3: Whiteboarding the Property Tax System (Intermediate/Advanced)

If asked to design the actual system described in the job listing, here is how you build the Entity-Relationship (ER) model.

Scenario: *Track properties, multiple owners per property, annual tax bills, and lawsuits filed for non-payment.*

1. **Property & Owner (M:M):** Since one property can have multiple owners, and one person can own multiple properties, we use a junction table: `PropertyOwner`.
2. **Property & TaxBill (1:M):** A property gets many annual tax bills (2022, 2023, 2024). Foreign Key (`PropertyID`) goes on the `TaxBill` table.
3. **TaxBill & Lawsuit (1:1 or 1:M):** If a bill isn't paid, a lawsuit is filed. Foreign Key (`BillID`) goes on the `Lawsuit` table pointing back to the `TaxBill`.

Property Tax ER Diagram Scenario

(When drawing this on your interview, make sure to explicitly show the lines connecting Primary Keys to Foreign Keys).

Level 4: SQL Programming & Optimization (Advanced)

A. Advanced SQL Retrieval (Joins)

For this role, you must know how to retrieve data traversing across these tables using SQL `JOIN`s.

```
-- Advanced: Find all properties with Open Lawsuits and get owner details.
SELECT
    P.Address,
    TB.AmountDue,
    O.FirstName + ' ' + O.LastName AS OwnerName,
    L.Status AS LawsuitStatus
FROM Lawsuit L
JOIN TaxBill TB ON L.BillID = TB.BillID               -- Connect Lawsuit to Bill
JOIN Property P ON TB.PropertyID = P.PropertyID       -- Connect Bill to Property
JOIN PropertyOwner PO ON P.PropertyID = PO.PropertyID -- Connect to junction table
JOIN Owner O ON PO.OwnerID = O.OwnerID                -- Connect junction to Owner
WHERE L.Status = 'Open'
ORDER BY TB.AmountDue DESC;
```

B. Stored Procedures (Very relevant to JD)

A Stored Procedure (SP) is pre-compiled SQL stored on the database server.

> *[!IMPORTANT]* ***Why are Stored Procedures critical for this role?***
>
> 1. ***Performance:*** *The Database engine caches the execution plan, so it runs much faster than sending ad-hoc query strings from an application.*
> 2. ***Security:*** *It protects against SQL Injection. Users or Applications only need `EXECUTE` permission on the procedure, rather than root `SELECT/UPDATE` permissions on the actual tables.*
> 3. ***Encapsulation:*** *Reduces complex logic and multi-step transactions down to a single callable standard.*

```
-- Creating an advanced stored procedure with Try/Catch error handling & Transactions
CREATE PROCEDURE [dbo].[ProcessTaxPayment]
    @BillID INT,
    @PaymentAmount DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON; -- Prevents extra 'row affected' network messages (optimization)
```

```sql
    BEGIN TRY
        BEGIN TRANSACTION; -- Start a transaction for safety

        -- Logic to update bill
        UPDATE TaxBill
        SET AmountDue = AmountDue - @PaymentAmount,
            IsPaid = CASE WHEN AmountDue - @PaymentAmount <= 0 THEN 1 ELSE 0 END
        WHERE BillID = @BillID;

        -- (You could also INSERT into a PaymentHistory table here safely)

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- If anything fails (e.g., server crash halfway through), rollback everything
        ROLLBACK TRANSACTION;
        THROW; -- Re-throw the error up to the application
    END CATCH
END;
```

C. Query Optimization (Performance Tuning)

When querying millions of tax or court records, performance tuning is key.

SQL Index Optimization Analogy vs Table Scans

1. **Table Scan (Bad):** Reading every single row in a table to find a match (like reading a book line by line from page 1 to find a specific word). Very slow!
2. **Index Seek (Good):** Using an index structure (like a B-Tree) to jump immediately to the correct data (like using the alphabetical index at the back of a textbook).
3. **How to Optimize:**
    - **Always index Foreign Keys** (e.g., `PropertyID` in the `TaxBill` table) and columns used frequently in `WHERE` or `ORDER BY` clauses. Primary Keys are indexed automatically.
    - Read the **Execution Plan** in SQL Server Management Studio (SSMS) to identify bottlenecks.
    - **Avoid** `SELECT *`. Explicitly naming only the columns you need reduces network payload overhead and server memory limits.

🎯 Quick-Reference Cheat Sheet

Essential SQL Commands

- `SELECT` : Retrieves data. (e.g., `SELECT Column1 FROM Table1`)
- `INSERT INTO` : Adds new rows. (e.g., `INSERT INTO Table1 (Col1) VALUES ('Hello')`)
- `UPDATE` : Modifies existing data. Always use with a `WHERE` clause! (e.g., `UPDATE Users SET Status = 'Active' WHERE ID = 1`)
- `DELETE FROM` : Removes rows. Always use with `WHERE` !
- `GROUP BY` : Groups aggregate data (used with `COUNT` , `SUM` , `AVG`).
- `HAVING` : Like a `WHERE` clause, but is used specifically to filter grouped aggregate data.

Joins Simplified

- `INNER JOIN` : Returns records that have matching values in *both* tables.
- `LEFT JOIN` : Returns all records from the left table, and the matched records from the right table (or NULL if no match).
- `RIGHT JOIN` : Returns all from the right table, and matches from the left.
- `FULL OUTER JOIN` : Returns all records when there is a match in either left or right table.

MS SQL Server Terminology

- **T-SQL**: Transact-SQL, the Microsoft extension to standard SQL. Brings variables, IF/ELSE logic, and TRY/CATCH blocks.
- **Execution Plan**: A visual representation of operations the DB engine performs to execute a query. Used to find performance bottlenecks.
- **Table Scan**: The database reads a table sequentially from top to bottom. (A red flag for large tables).
- **Index Seek**: The database uses an index B-Tree to instantly find the data row. (Highly optimized).
- **Clustered Index**: Sorts and physically stores the data rows based on their key values. You can only have ONE per table (usually the Primary Key).
- **Non-Clustered Index**: A separate structure from the data rows that contains pointers to the data. You can have many of these on a table (usually applied to Foreign Keys).

Agile & Developer Terms (From Job Description)

- **Jira**: A popular project management tool used to track tickets, bugs, and tasks.
- **Scrum Master**: The person who organizes the team's workflow, facilitates meetings, and removes roadblocks.
- **MVP (Minimum Viable Product)**: The earliest, simplest version of a feature that is usable enough to provide value and gather feedback.
- **API (Application Programming Interface)**: How two different systems (like Texas E-file and Lawbase) talk to each other and exchange data securely.

📝 20 Potential Interview Questions & Answers

These questions are specifically tailored to the unique requirements outlined in the Harris County Attorney's Office job description.

Part 1: Agile, Work Style, & Behavioral

Q1. We are a small team without a QA department. How do you ensure your code is bug-free before deployment?

> *Answer: "Since there is no dedicated QA, I rely heavily on rigorous unit testing. I write edge cases for my input parameters, test my stored procedures against edge-case data (like NULL fields or negative values), and deploy to a sandbox/development database to test the UI interactions before calling it an MVP."*

Q2. You are given a ticket in Jira, but the requirements are vague. What is your next step?

> *Answer: "I don't wait for someone to tell me what to do. The job description mentioned working closely with Business Systems Analysts (BSA), so I would proactively message the BSA or the main stakeholders to clarify the business logic, document the new requirements, and update the Jira ticket myself."*

Q3. How do you manage your time when given a loose deadline but no strict daily micromanagement?

> *Answer: "I am a self-starter. I break the project down into smaller functional chunks, log those steps in Jira, and update the board daily. If I hit a roadblock that I can't solve within an hour of researching, I reach out to a senior*

> *developer or the Scrum Master to ensure I don't block the sprint."*

Q4. We use a low-code platform called "Lawbase." Are you comfortable learning a proprietary tool even if the exact skills might not translate heavily to your next job?

> *Answer: "Absolutely. Every organization uses custom workflows or legacy systems. Learning Lawbase shows my adaptability and system-level thinking. The core logic of calling SQL functions and designing automated workflows will always make me a better engineer, regardless of the wrapper tool."*

Q5. Explain a time you had to take the lead on a technical problem.

> *(Use a personal project or class project where you hit a bug and solved it by reading documentation or taking architectural ownership).*

Part 2: Database Design Foundation

Q6. What is the difference between a Primary Key and a Foreign Key?

> *Answer: "A Primary Key uniquely identifies a row identically within its own table. A Foreign Key is a column in a table that points to a Primary Key in another table, establishing a relationship between the two."*

Q7. How do you represent a Many-to-Many relationship in a relational database?

> *Answer: "You cannot represent it directly. You must create an intermediary table, usually called a junction or intersection table. For example, to link Properties and Owners, I would create a `PropertyOwner` table that stores the `PropertyID` and `OwnerID`."*

Q8. Can you explain the concept of Database Normalization to a non-technical person?

> *Answer: "Imagine a messy Excel file where you type a client's address a hundred times because they have a hundred invoices. If they move, you have to find and update 100 rows, risking a typo. Normalization splits that into two neat lists—one for clients, one for invoices—linked by an ID. If they move, you update it once, and all invoices instantly reflect the correct address."*

Q9. What is NULL in a database? Is it the same as zero or a blank string?

> *Answer: "No. NULL represents the absence of a value or an 'unknown' state. It is not zero, and it is not an empty string. Any math operation involving NULL results in NULL."*

Part 3: Intermediate to Advanced SQL & T-SQL

Q10. What is a Stored Procedure, and why do we prefer it over standard SQL strings?

> *Answer: "A stored procedure is a prepared SQL code block saved on the database server. We prefer them for performance (because the execution plan is cached), for security (preventing SQL injection and abstracting table access), and for encapsulating complex logic."*

Q11. We need to query for a list of all Properties, even if they DO NOT have any lawsuits against them yet. Which JOIN would you use?

> *Answer: "I would use a `LEFT JOIN` taking `Property` as the left table and joining `Lawsuit`. This guarantees every property is returned, and properties without a lawsuit will just show `NULL` in the lawsuit columns."*

Q12. What is the difference between the `WHERE` clause and the `HAVING` clause?

> *Answer: "The `WHERE` clause filters rows before any grouping occurs. The `HAVING` clause filters data after it has been grouped by the `GROUP BY` clause (e.g., `HAVING SUM(Amount) > 1000`)."*

Q13. How would you handle an error occurring halfway through a Stored Procedure that modifies three different tables?

> *Answer: "I would use a `BEGIN TRANSACTION` block at the start, wrapped inside a `TRY...CATCH` block. If an error occurs during the execution, the code falls into the `CATCH` block where I would call `ROLLBACK TRANSACTION` to revert all partial changes and keep the database in a consistent state."*

Q14. How do you find duplicate records in a table?

> *Answer: "Using `GROUP BY` and `HAVING`. For example: `SELECT Email, COUNT(*) FROM Users GROUP BY Email HAVING COUNT(*) > 1;`."*

Part 4: Query Optimization & Performance

Q15. An attorney complains that generating their daily report is taking 3 minutes. How do you troubleshoot the underlying SQL query?

> *Answer: "First, I would run the query in SQL Server Management Studio and generate the 'Actual Execution Plan.' I would look for high-cost operations, specifically 'Table Scans' or 'Index Scans'. From there, I can determine if a specific `WHERE` clause or `JOIN` needs a new Index."*

Q16. Explain the difference between an Index Seek and a Table Scan.

> *Answer: "A Table Scan means the database has to read every single row in the table to find matches, which is incredibly slow for large datasets. An Index Seek means the database uses a B-Tree index structure to instantly jump to the exact location of the data, which is highly efficient."*

Q17. What is the difference between a Clustered and a Non-Clustered Index?

> *Answer: "A Clustered Index physically sorts and stores the table's rows on disk by the index key, so you can only have one per table (usually the Primary Key). A Non-Clustered index is a separate structure that holds pointers to the physical data rows, and you can have many of them on a table."*

Q18. Are there any downsides to adding indexes to every column to speed up SELECT queries?

> *Answer: "Yes! While indexes drastically speed up `SELECT` statements, they slow down `INSERT`, `UPDATE`, and `DELETE` statements because the database must pause to rewrite the index B-Tree every time the data changes. Too many indexes will cripple your write performance."*

Q19. What is a SQL View?

> *Answer: "A View is a virtual table based on the result-set of an SQL statement. It doesn't store data itself; it just provides a saved, reusable query that acts like a table, which is great for simplifying complex joins."*

Q20. The job mentions establishing API integrations with 'E-file Texas by Tyler Technologies'. How would your SQL Server database interact with an external API?

> *Answer: "SQL Server can interact with APIs sometimes via specialized CLR (Common Language Runtime) stored procedures, but the modern best practice is to write a script or small microservice (like C# or Python) or use the 'Lawbase' wrapper tool to make the HTTP REST calls, retrieve the JSON data, and then call a standard Stored Procedure to insert that parsed data securely into the database."*